CSC 6302 Computer Vision

Practicals 1–7 — My Walkthrough Pack

Munyeme Sinyani · University of Zambia · 2025/26

Generated 11 September 2026

This PDF is my own study notes — written like I am explaining to a confused classmate who missed the lab. Same vibe as Assignment 1: a computer does not see a picture, it sees a grid of numbers. Every practical here is about what those numbers mean and what we do to them.

How to use this pack

Think of this repo as three layers: (1) runnable scripts that produce report figures, (2) small Gradio apps where the brief asks for a demo, and (3) this PDF that tells you why each line exists. Do not submit this PDF blindly — use it to understand, then write your own report in your own words.

Suggested order

- Create a venv and `pip install -r requirements.txt`.
- Run `python shared/make_demo_images.py` once (synthetic stand-ins).
- Run each `pracN/run_pracN.py` (or `train_prac4.py`) to fill `pracN/report/figures/`.
- Open the PNGs while reading the matching section below — the pictures are the proof.
- Replace demo images with your own phone photos before final submission.
- Re-run this script to refresh the master PDF after new figures exist.

General course instructions (distilled)

The arc across the semester

The practicals are deliberately stacked. We start where Assignment 1 left off — pixels as numbers — then climb: edges and morphology (Prac 1), interest points and stitching (Prac 2), detection (Prac 3), learning features (Prac 4), segmentation (Prac 5), motion and geometry (Prac 6), and finally a deployed capstone (Prac 7). Each week assumes you still remember the previous layer.

From-scratch rules (non-negotiable)

- When the brief says implement X yourself, calling the library one-liner scores zero — same rule as Assignment 1 banning `cv2.Sobel` for your own Sobel task.
- In this repo, the from-scratch pieces are: PSNR, 2-D convolution + Sobel magnitude, Harris R, IoU, and Kalman predict/update. I must be able to explain every line in a viva.
- OpenCV/PyTorch are fine for glue (load image, show result, train a head) unless the task explicitly forbids it.

Submission conventions

- Individual practical repo name: `csc6302-pracN-SINYANI` (N = 1…7).
- Capstone team repo: `csc6302-capstone-SINYANI-PARTNER`.
- Typical hand-in: code + short PDF report + figure folder + link to Hugging Face Space where required.
- Zip naming often follows `SINYANI_ComputerNumber_PracticalN.zip` — check the LMS post for the exact string.
- Commit little and often; markers like to see history, not one giant upload on deadline night.

AI use — declare honestly

I used Cursor (Composer) to scaffold pipelines, debug OpenCV/PyTorch glue, and draft this walkthrough. That is allowed only if I declare it and still understand the work. The intellectual core — problem framing, parameter choices, what the figures prove — must be mine. Copy-pasting AI text into the marked report without acknowledgement is academic dishonesty.

Practical 1 — Seeing Like a Machine

Why this practical exists

Before deep learning magic, we need the boring truth: colour is three number channels, noise is just wrong numbers, and 'finding edges' means asking where brightness changes fast. This practical connects Assignment 1 (images as arrays) to the pipelines we will reuse all semester — threshold, filter, measure.

Task-by-task (baby steps)

Task 1 — Colour spaces (HSV / LAB)

RGB is how the camera stores bytes; HSV splits hue (what colour) from saturation and value (how strong / how bright). LAB is closer to human perception. We split channels so we can pick objects by hue — e.g. mask the red region with `cv2.inRange` on H, because red wraps around 0° in hue.

Task 2 — Noise and PSNR

We corrupt a clean grayscale image with Gaussian noise (add random values) and salt-and-pepper (random pixels to 0 or 255). Then we try box, Gaussian, and median filters. PSNR tells us how close the restored image is to the clean original — higher is better. We implement PSNR ourselves because the brief wants the maths, not `cv2.PSNR`.

Task 3 — Sharpening

Sharpen = original + amount × (original − blurred). The blur estimate removes low frequencies; adding the difference back boosts edges. Too much amount creates ugly halos — that is a feature, not a bug, for the report discussion.

Task 4 — Edges (own Sobel + Canny + LoG)

Sobel estimates partial derivatives Gx and Gy with 3×3 kernels, then magnitude $\sqrt{(Gx^2+Gy^2)}$. I wrote `conv2d` with loops so I can point at each multiply-add in a viva. Canny is the industrial version: blur, gradient, non-max suppression, hysteresis thresholds. LoG (Laplacian of Gaussian) finds zero-crossings — another edge story.

Task 5 — Morphology + counting

Threshold → erode/dilate/open/close cleans a binary mask. Connected components counts separate blobs — our coin counter. The CLI in `process.py` wraps the same ops for marker testing.

Core formula / snippet to know

```
def psnr(clean, restored):
    mse = np.mean((clean.astype(float) - restored.astype(float)) ** 2)
    if mse == 0:
        return float("inf")
    return 10.0 * np.log10((255.0 ** 2) / mse)

# Gx and Gy are 3x3 Sobel kernels; convolve, then:
mag = np.sqrt(gx ** 2 + gy ** 2)
```

How marks usually work

- Correct colour-space discussion with figures for each channel.
- PSNR table comparing filters — explain why median wins on salt-and-pepper.
- Working from-scratch Sobel with comparison to OpenCV reference.

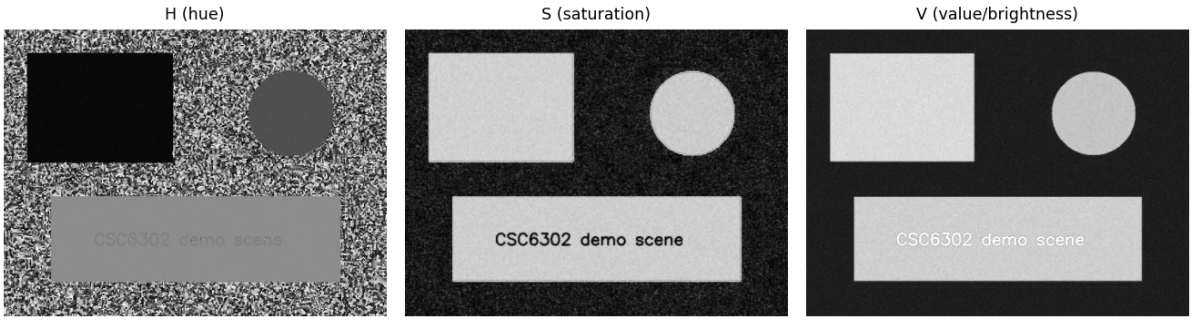
- Morphology pipeline with defensible object count.
- CLI runs all ops; report explains failures (halos, broken edges).

How to run

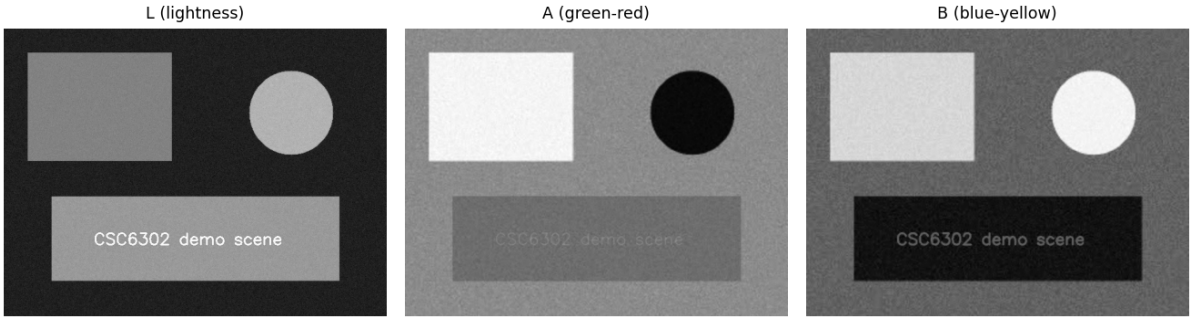
```
python shared/make_demo_images.py
python prac1/run_prac1.py
# CLI examples:
python prac1/process.py --input shared/images/colourful.jpg --op pick --output out.png
python prac1/process.py --input shared/images/coins.jpg --op count --output count_vis.png
```

Figures to inspect

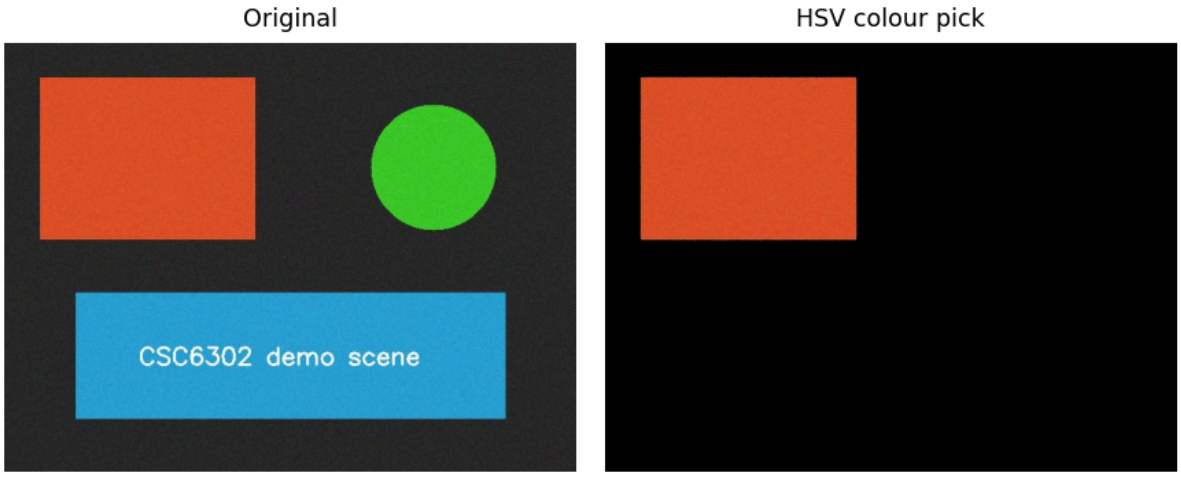
- t1_hsv_channels.png, t1_lab_channels.png, t1_picker.png — colour space intuition.
- t2_noise.png, t2_restored.png, t2_psnr.csv — denoising evidence.
- t3_sharpen.png — halo trade-off at amount=2.0.
- t4_sobel.png, t4_canny.png, t4_log.png — edge detectors side by side.
- t5_morph.png, t5_count_vis.png — morphology and count.



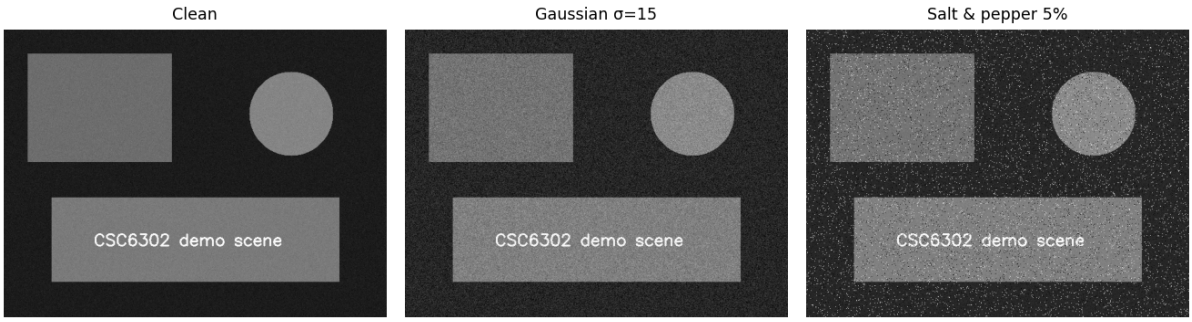
t1_hsv_channels.png



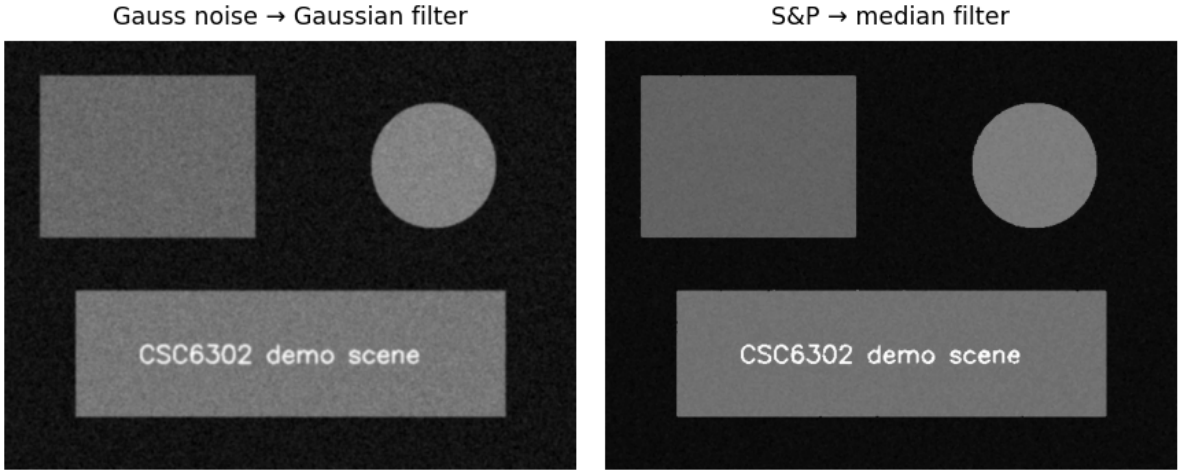
t1_lab_channels.png



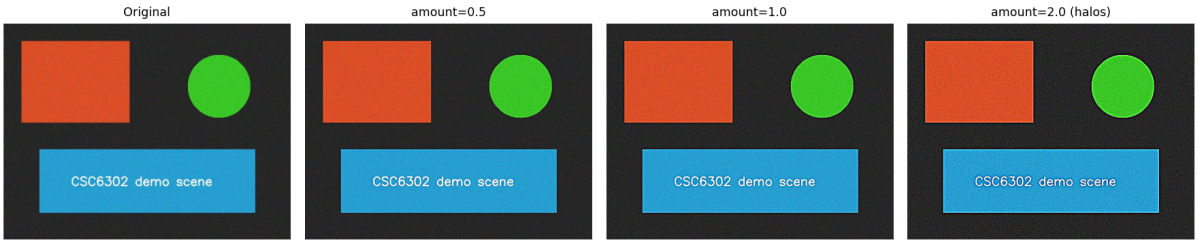
t1_picker.png



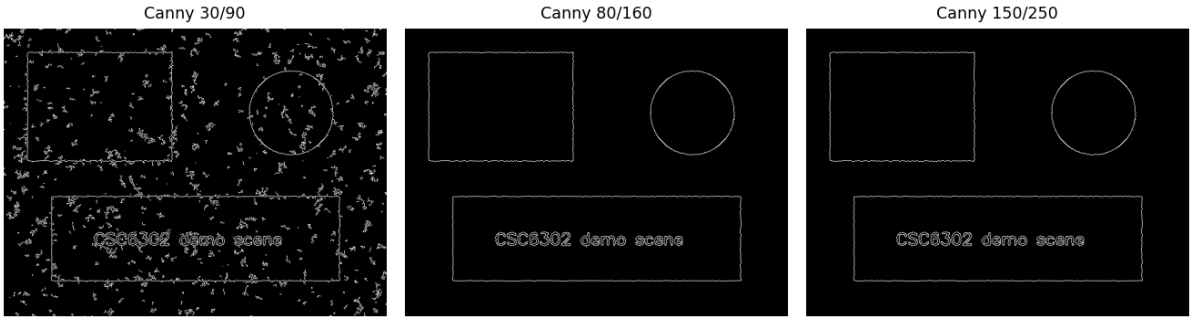
t2_noise.png



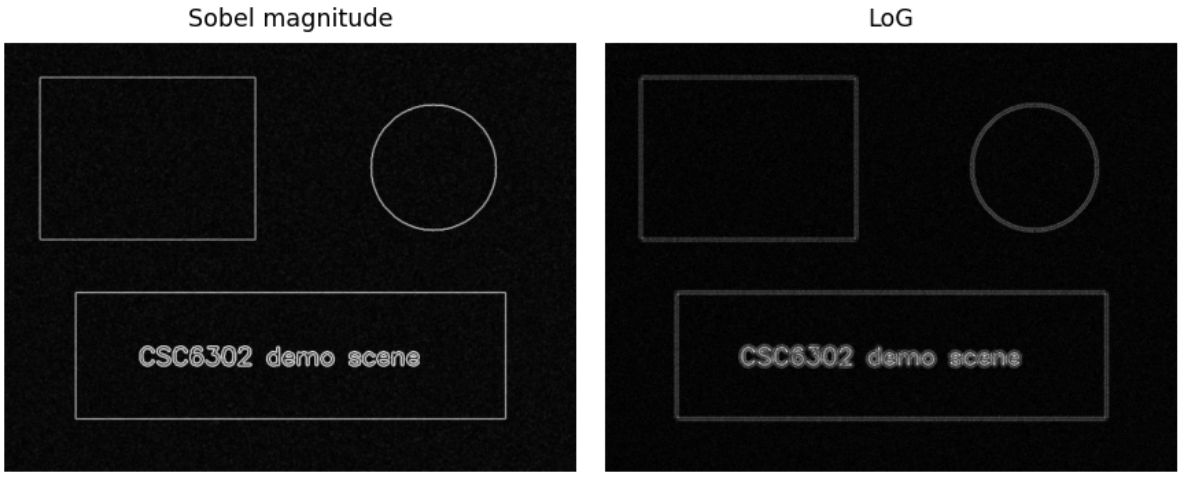
t2_restored.png



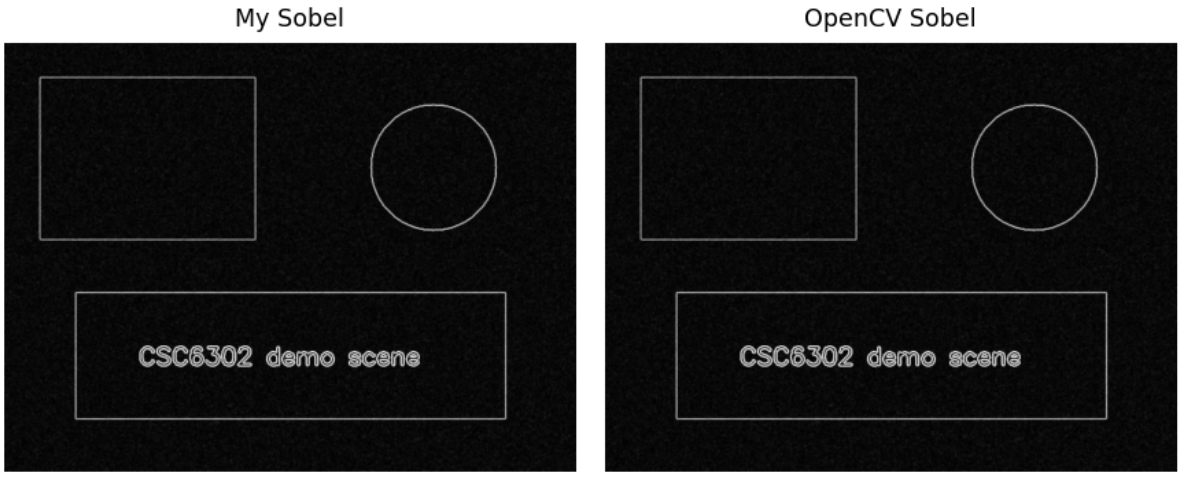
t3_sharpen.png



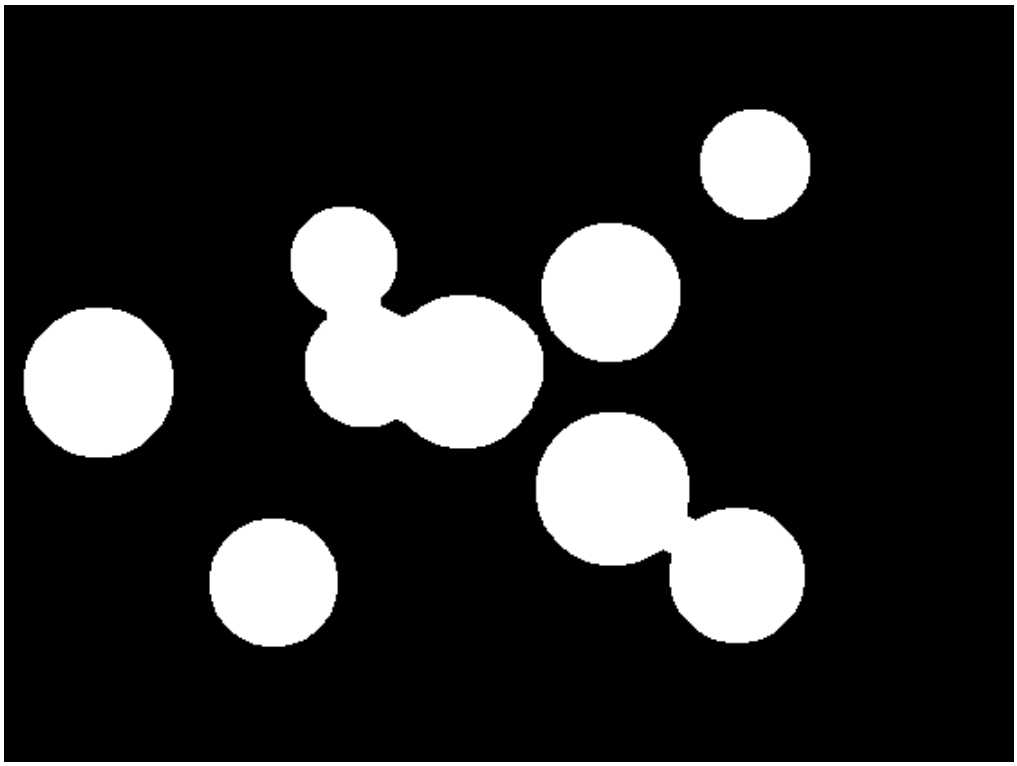
t4_canny.png



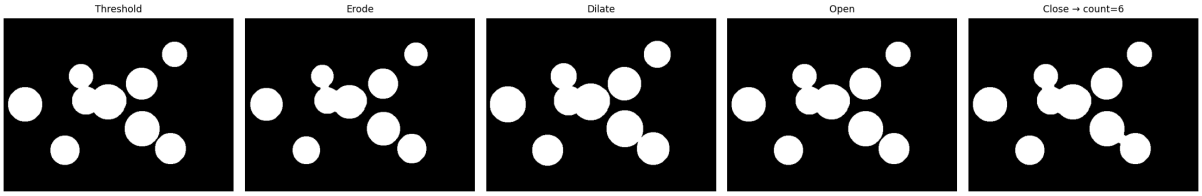
t4_log.png



t4_sobel.png



t5_count_vis.png



t5_morph.png

Practical 2 — Interest Points, Matching, Panorama

Why this practical exists

How does Google Photos stitch your camera roll? It finds the same corners in two images, matches descriptors, rejects outliers with RANSAC, then warps. This week is that story — from Harris corners to homography.

Task-by-task (baby steps)

Task 1 — Corner detectors

Corners are points where gradient changes in all directions (Harris R). Flat regions and straight edges score badly. We implement Harris from scratch and compare counts to Shi-Tomasi and FAST on checkerboard vs natural texture.

Task 2 — Descriptor invariance

ORB (binary, fast) vs SIFT (float, heavier) under rotation and scale. We measure repeatability: after warping, what fraction of keypoints match within a few pixels? SIFT usually wins on hard transforms; ORB is the budget option.

Task 3 — Matching with ratio test

Nearest-neighbour matching alone is noisy. Lowe's ratio test keeps a match only if the best descriptor is clearly better than the second-best — kills ambiguous matches.

Task 4 — Panorama / homography

Matched points give a homography H via RANSAC. Warp image A into B's plane, blend. Inlier count tells you if the stitch is trustworthy.

Task 5 — Robustness

Blur, darkness, JPEG compression — repeat matching. Good features survive moderate degradation; bad lighting breaks naive pipelines. Document where yours fails.

Core formula / snippet to know

```
# After Gaussian-smoothed Ix2, Iy2, Ixy:
R = (Ix2 * Iy2 - Ixy ** 2) - k * (Ix2 + Iy2) ** 2
# Keep local maxima of R above a threshold → corner points
```

How marks usually work

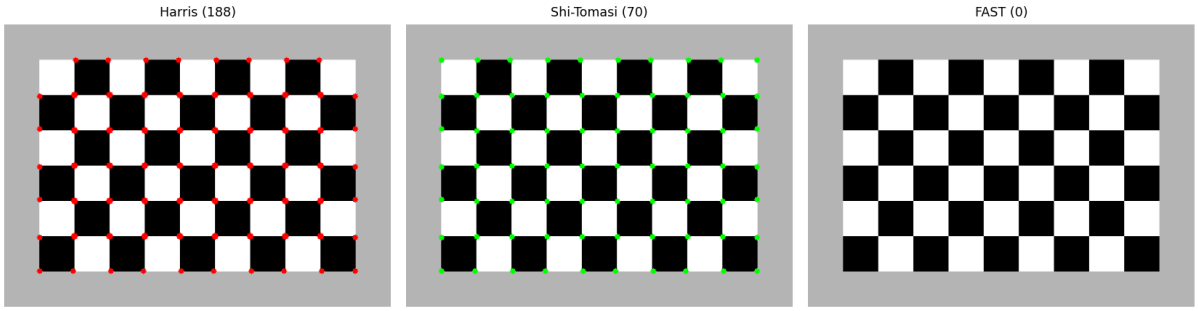
- From-scratch Harris with sensible corner overlay figures.
- Repeatability table (ORB vs SIFT) with short interpretation.
- Ratio-test match visualisation; explain false matches without ratio.
- Panorama with RANSAC inlier count reported.
- Robustness CSV under at least three degradations.

How to run

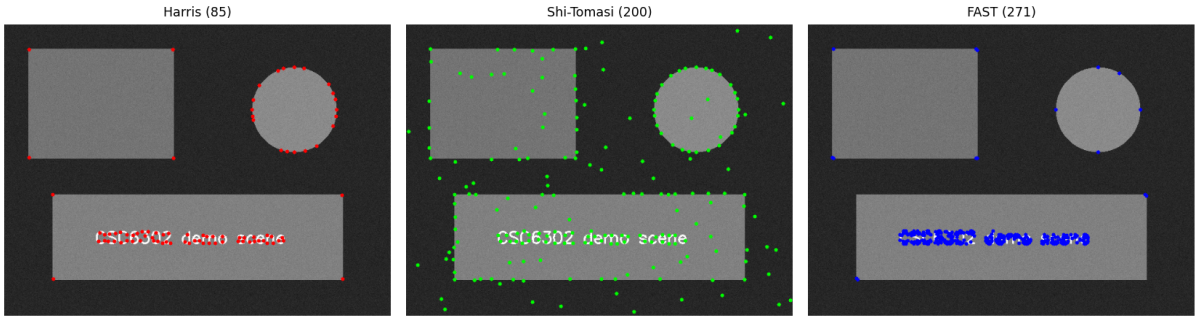
```
python prac2/run_prac2.py
```

Figures to inspect

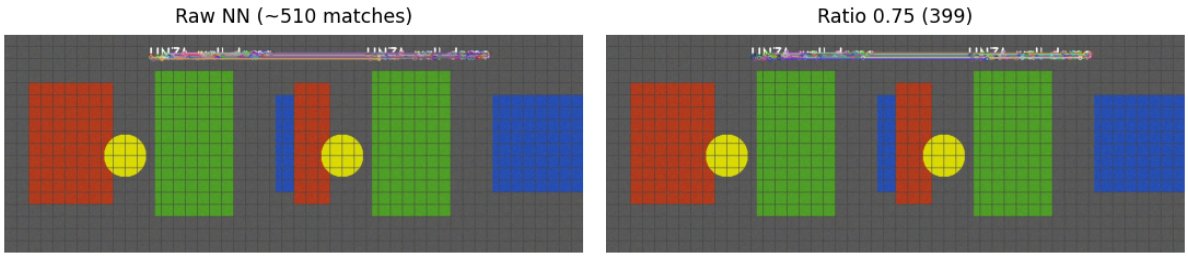
- t1_checkerboard_corners.png, t1_natural_corners.png — detector comparison.
- t3_matches.png — raw vs ratio-filtered matches.
- t4_panorama.png, t4_panorama_wide.png — stitch results.
- t2_repeatability.csv, t5_robustness.csv — numbers for the report.



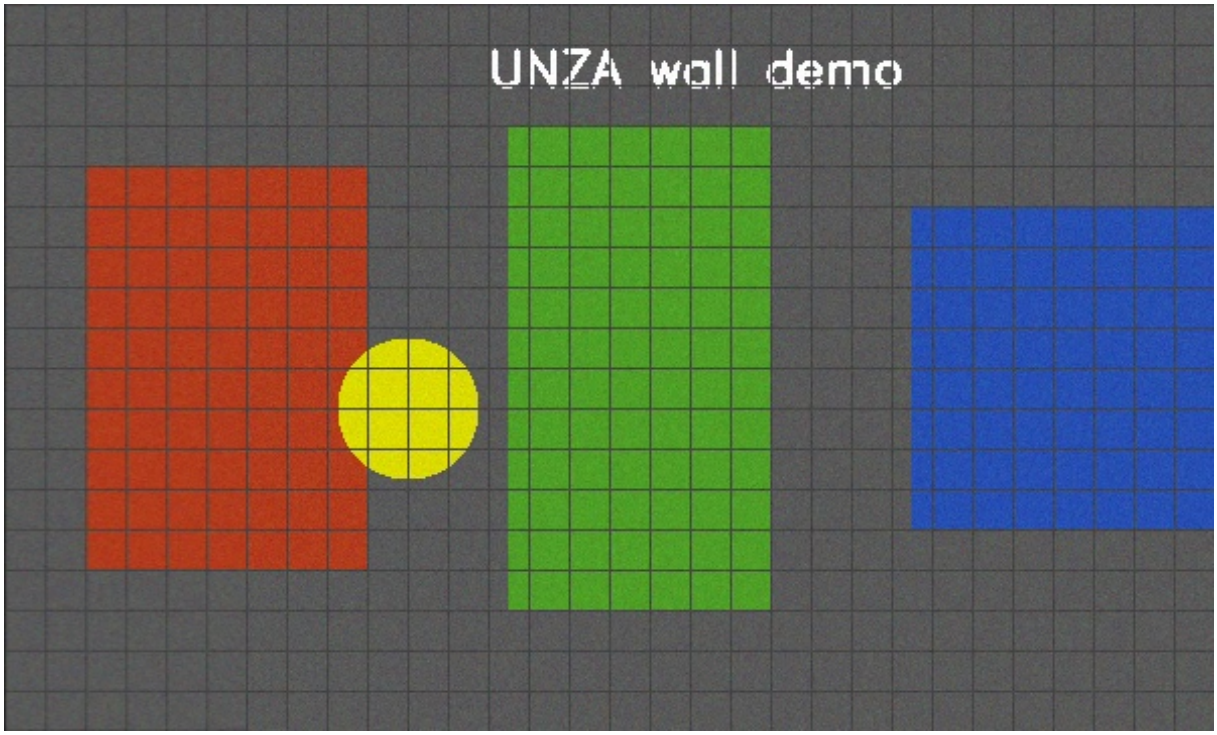
t1_checkerboard_corners.png



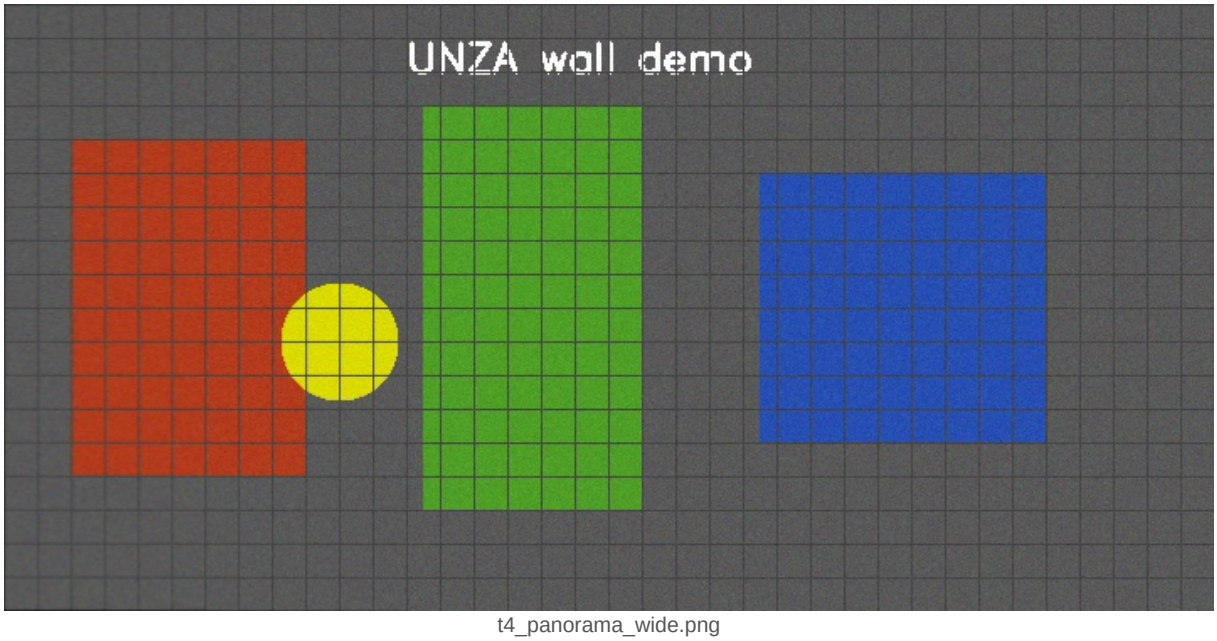
t1_natural_corners.png



t3_matches.png



t4_panorama.png



t4_panorama_wide.png

Practical 3 — Object Detection — Classical to YOLO

Why this practical exists

Detection = draw a box and a label. Classical methods (template, Haar) are fast and interpretable but brittle. Deep detectors (YOLO) generalise but need GPU and data. This practical is the before/after story every CV student needs.

Task-by-task (baby steps)

Task 1 — Template matching

Slide a template across the image; best correlation peak = location. Works when scale and rotation match; fails when they do not — we show that with a scaled scene.

Task 2 — Haar cascades

Viola-Jones scans rectangular Haar features at many scales. Parameters `scaleFactor` and `minNeighbors` trade speed vs false positives. We sweep them and annotate a short video.

Task 3 — YOLOv8

One forward pass predicts many boxes + classes. Confidence threshold study shows how low conf floods you with junk detections. Compare timing vs Haar on motion frames.

Core formula / snippet to know

```
# Template: cv2.matchTemplate(..., cv2.TM_CCOEFF_NORMED)
# Haar: cascade.detectMultiScale(gray, scaleFactor, minNeighbors)
# YOLO: model(img, conf=0.25)
```

How marks usually work

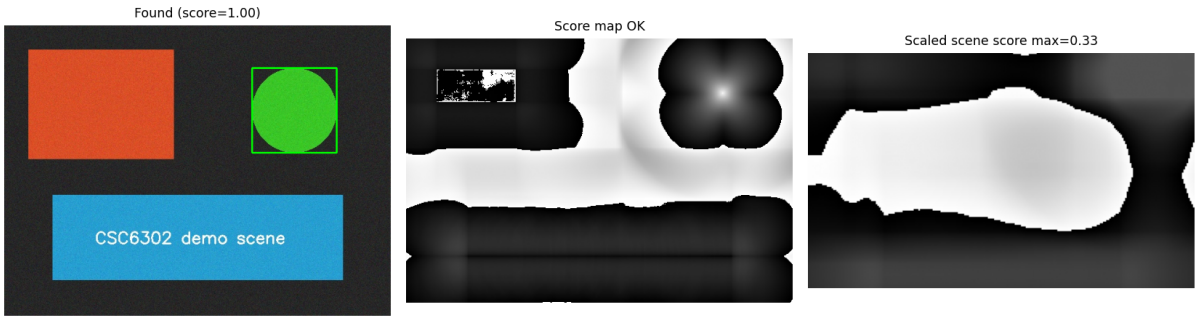
- Template success + intentional failure case explained.
- Haar parameter table and annotated still/video.
- YOLO detections at multiple conf values; timing comparison.
- Gradio app (`prac3/app.py`) demoing Haar vs YOLO.
- HF Space link if the brief requires hosted demo.

How to run

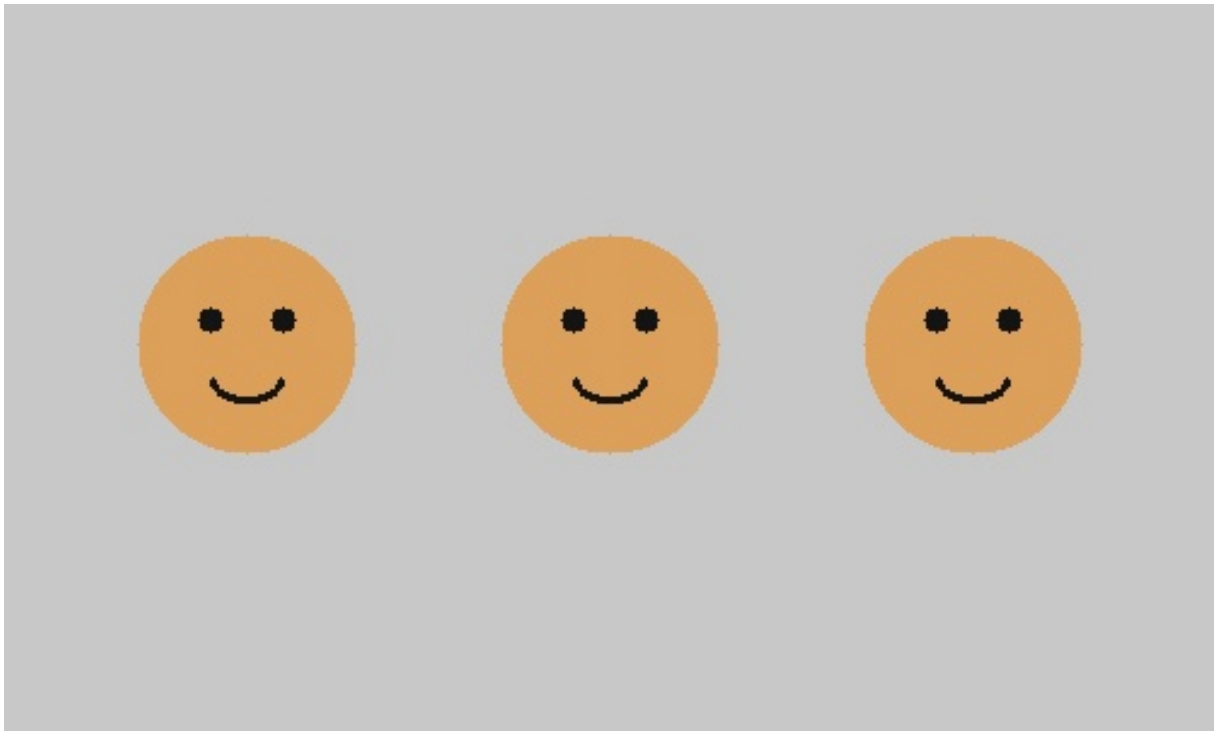
```
pip install ultralytics   # if not already
python prac3/run_prac3.py
python prac3/app.py
```

Figures to inspect

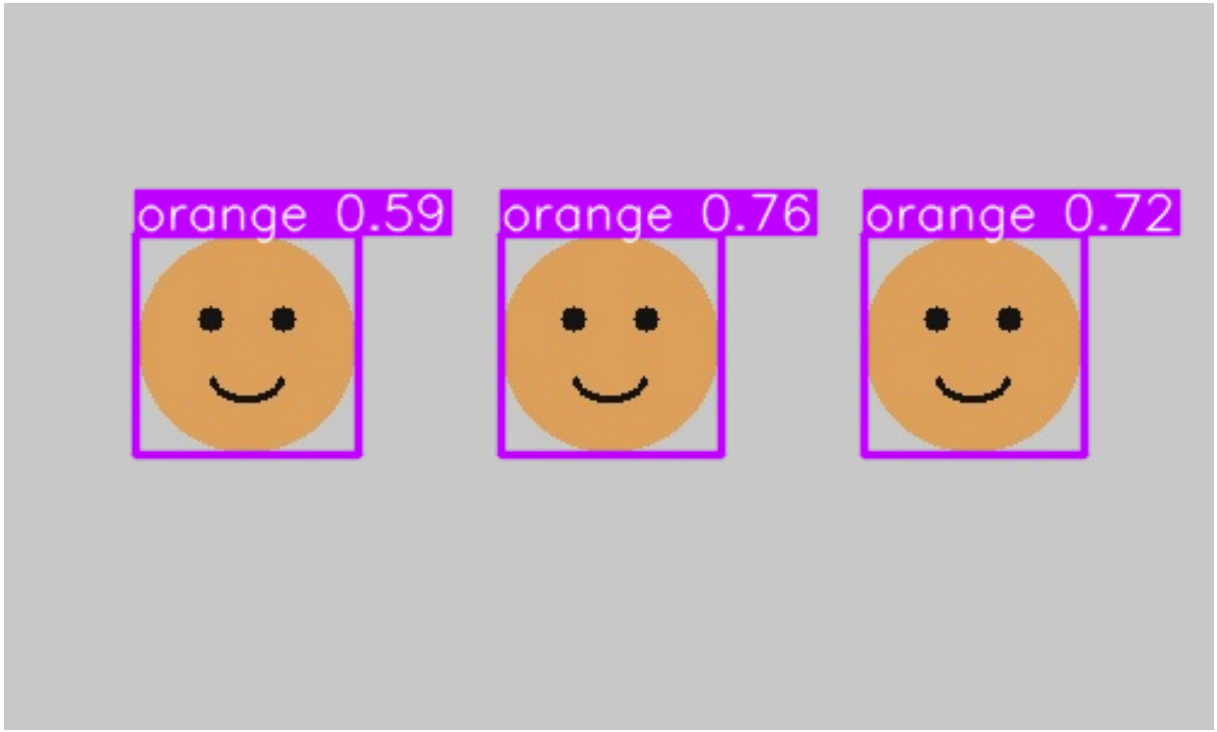
- t1_template.png — match vs scaled failure.
- t2_haar_group.png — face boxes (may be zero on synthetic group).
- t3_yolo_group.png, t3_yolo_street.png, t3_yolo_objects.png — YOLO overlays.
- t3_conf_study.csv, t3_timing.txt — numbers for discussion.



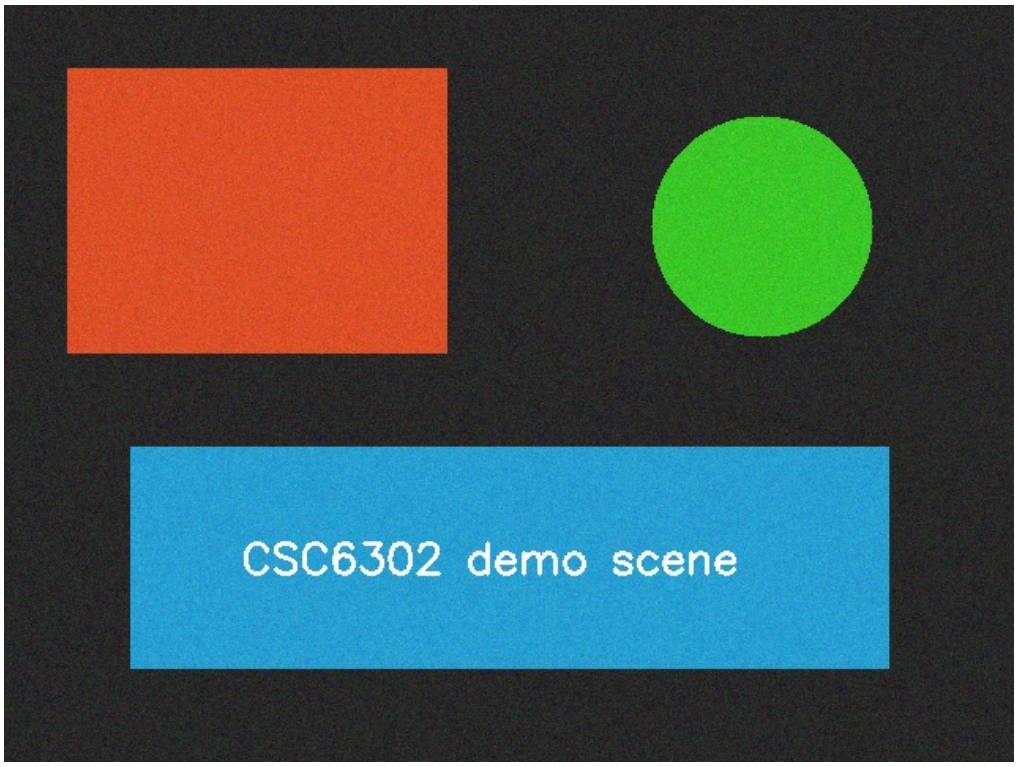
t1_template.png



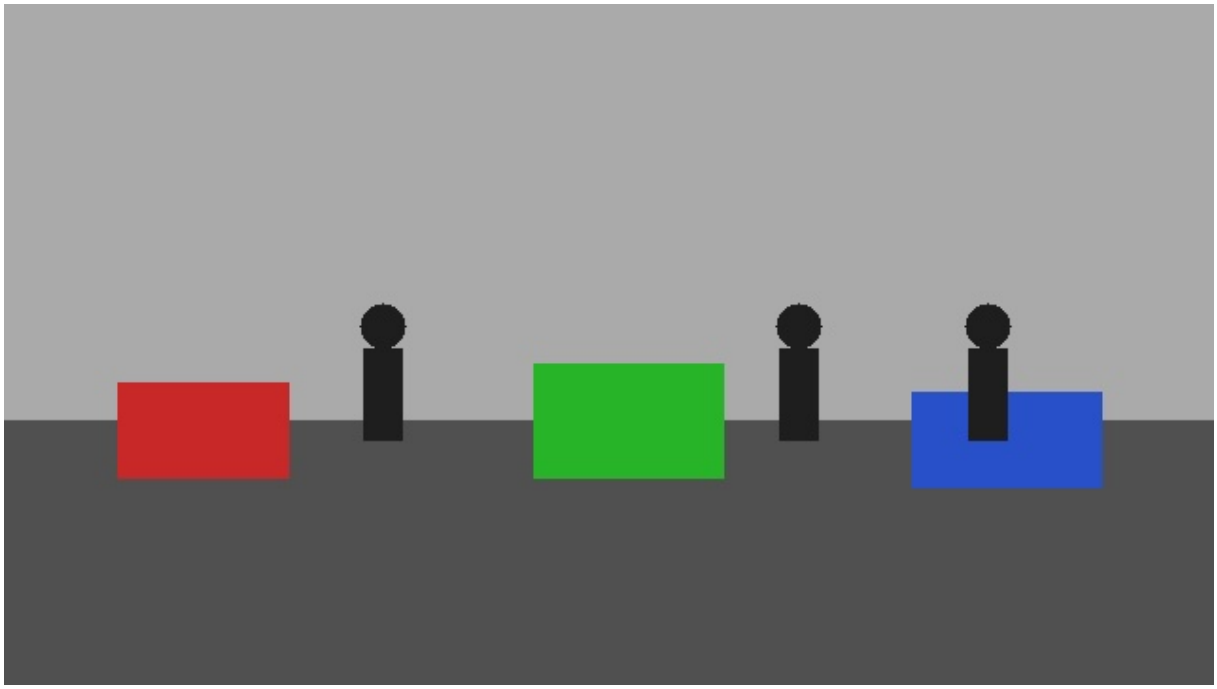
t2_haar_group.png



t3_yolo_group.png



t3_yolo_objects.png



t3_yolo_street.png

Practical 4 — CNNs and Transfer Learning

Why this practical exists

Hand-crafting features got us here; learning features gets us further. We train a tiny CNN from scratch, then swap in MobileNetV3 pretrained on ImageNet — freeze backbone, train head, then lightly fine-tune. Same story as CIFAR-10 in the brief, but we use a small leaf dataset locally so CPU training finishes in minutes.

Task-by-task (baby steps)

Task 1 — Scratch TinyCNN

Three conv blocks + pooling learn filters from random init. Expect lower accuracy and longer training — that is the point of comparison.

Task 2 — Transfer learning

ImageNet weights already encode edges/textures. We replace the classifier head for our 3 classes (healthy / rust / blight), train head only, then unfreeze last block for fine-tune.

Task 3 — Grad-CAM

Which pixels pushed the prediction? Backprop to a conv layer, weight activations by gradients, overlay heatmap. Sanity-check: model should look at leaf-ish regions, not background.

Core formula / snippet to know

```
# Transfer head swap (PyTorch):
# m.classifier[3] = nn.Linear(in_features, n_classes)
# train only m.classifier.parameters() first
```

How marks usually work

- Training curves for scratch vs transfer.
- Test accuracy + confusion matrix.
- Comparison JSON with training time — honest about CPU vs Colab GPU.
- Grad-CAM figures for correct and wrong predictions.
- Gradio / HF Space classifier app loading saved `model.pt`.

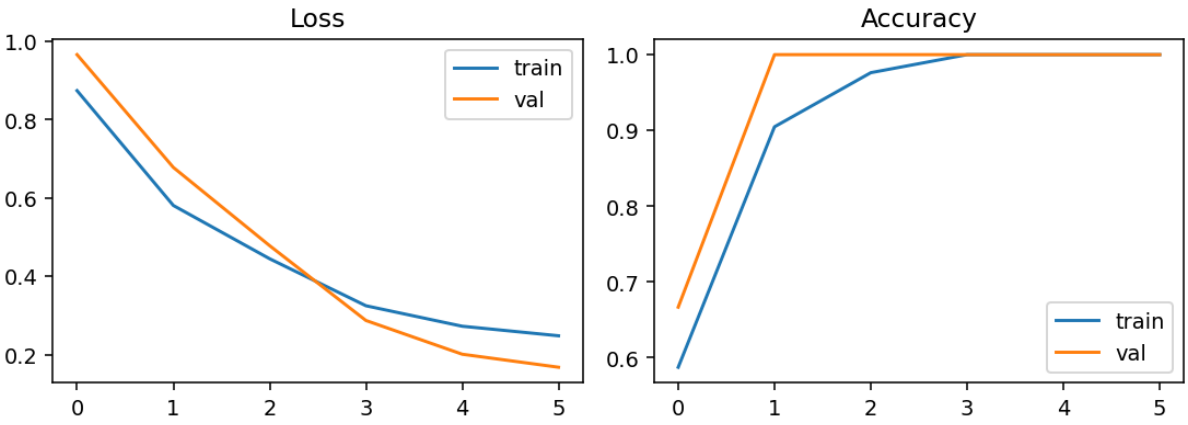
How to run

```
# CPU demo (minutes):
python prac4/train_prac4.py
python prac4/app.py

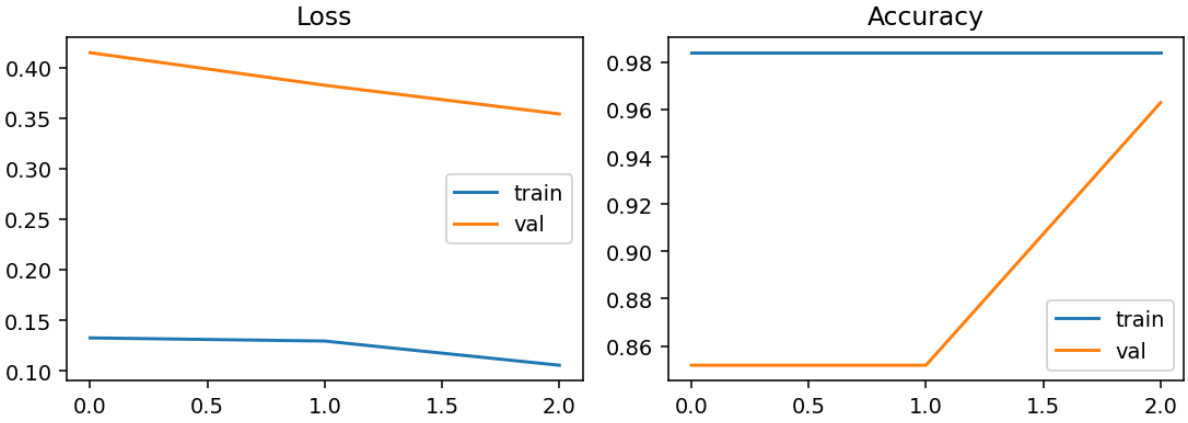
# For real CIFAR-10 marks: re-run notebook on Colab GPU
# with more epochs — same pipeline, bigger data.
```

Figures to inspect

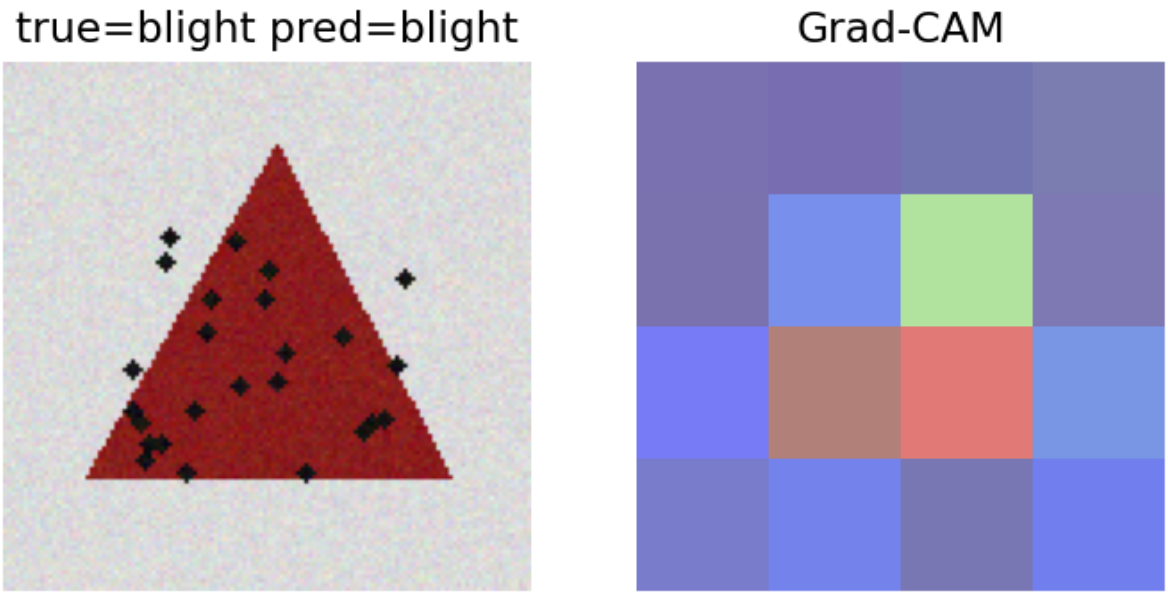
- t1_curves_scratch.png — scratch training loss/acc.
- t2_curves_finetune.png — fine-tune phase.
- t1_confusion.csv, t2_comparison.json — metrics.
- t3_gradcam_correct_*.png, t3_gradcam_miss_*.png — interpretability.



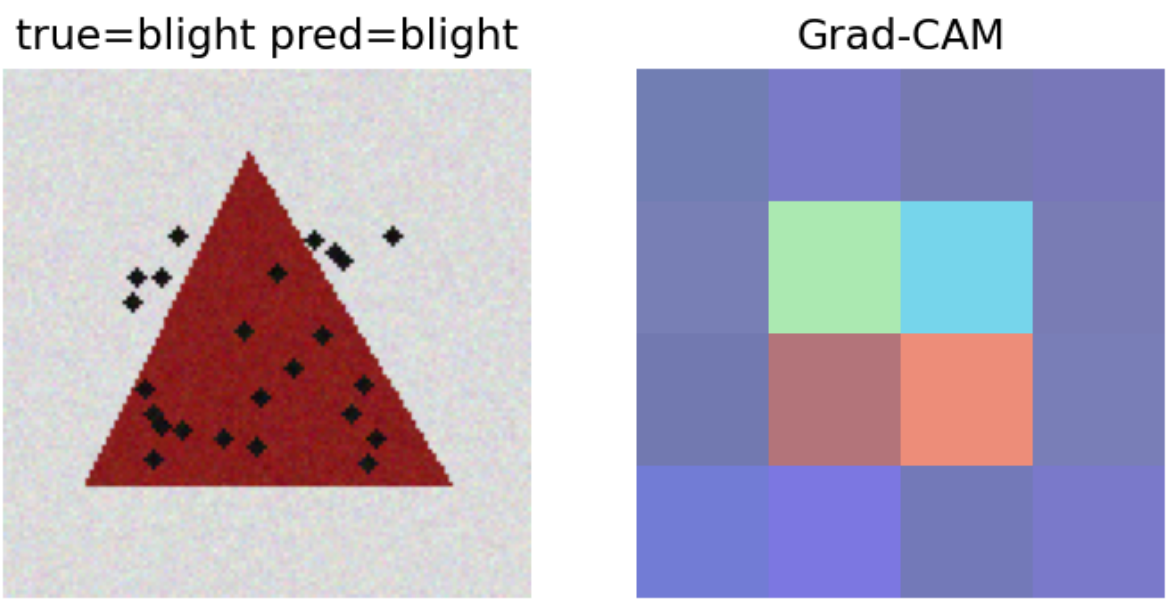
t1_curves_scratch.png



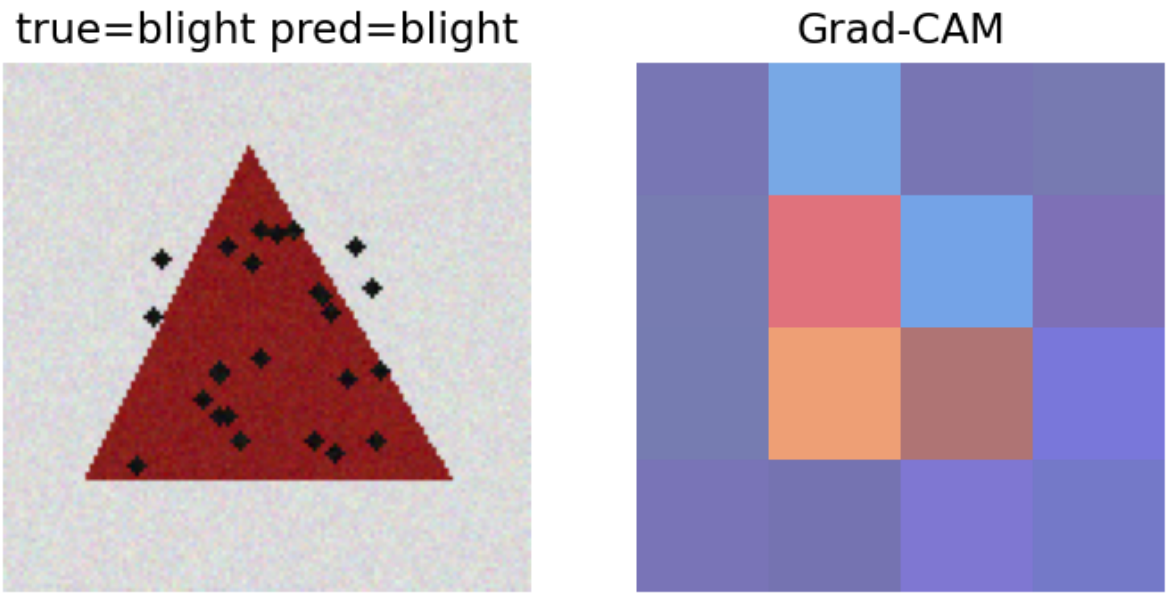
t2_curves_finetune.png



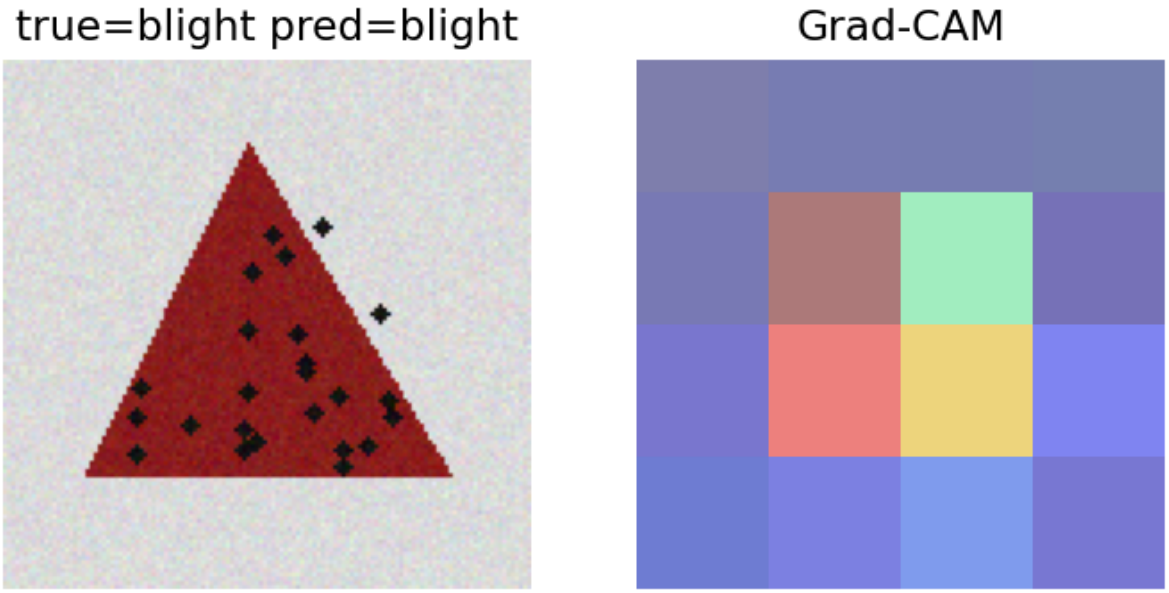
t3_gradcam_correct_0.png



t3_gradcam_correct_1.png



t3_gradcam_correct_2.png



t3_gradcam_correct_3.png

Practical 5 — Segmentation

Why this practical exists

Detection draws boxes; segmentation labels every pixel. We climb from thresholding through watershed and GrabCut to optional DeepLab / Mask R-CNN — classical vs deep, semantic vs instance.

Task-by-task (baby steps)

Task 1 — Thresholding + IoU

Fixed threshold, Otsu (automatic), adaptive mean/Gaussian. We score against a hand-made reference mask with IoU — intersection over union, implemented ourselves.

Task 2 — Watershed

Treat gradients like a landscape; flood from markers to split touching objects. Good for separated blobs (coins), bad when markers wrong.

Task 3 — GrabCut

Iterative graph cut with a rectangle (and optional scribble mask). More iterations = cleaner boundaries around foreground.

Task 4 — Deep segmentation (optional on CPU)

DeepLabV3 semantic labels every pixel; Mask R-CNN adds instance masks. Slow on CPU — report timing honestly.

Core formula / snippet to know

```
inter = np.logical_and(pred > 0, gt > 0).sum()
union = np.logical_or(pred > 0, gt > 0).sum()
iou = inter / union if union else 1.0
```

How marks usually work

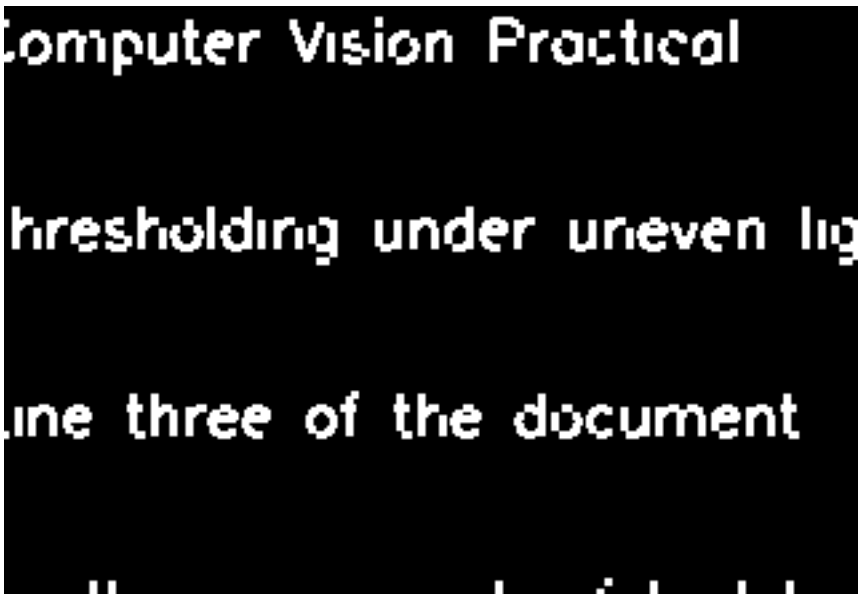
- IoU table for threshold methods with visual comparison.
- Watershed pipeline figure + object count note.
- GrabCut iterations vs mask hints.
- Deep vs classical timing if Task 4 runs.
- Gradio segmentation studio (`prac5/app.py`).

How to run

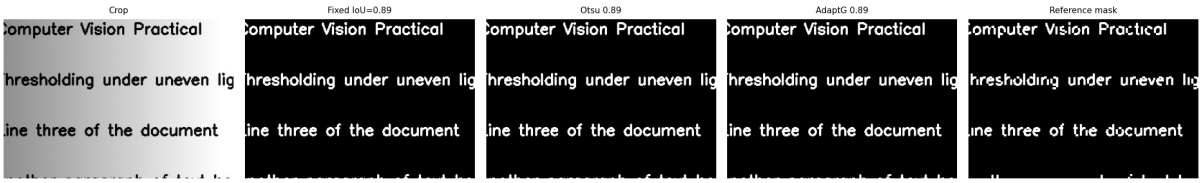
```
python prac5/run_prac5.py
python prac5/app.py
```

Figures to inspect

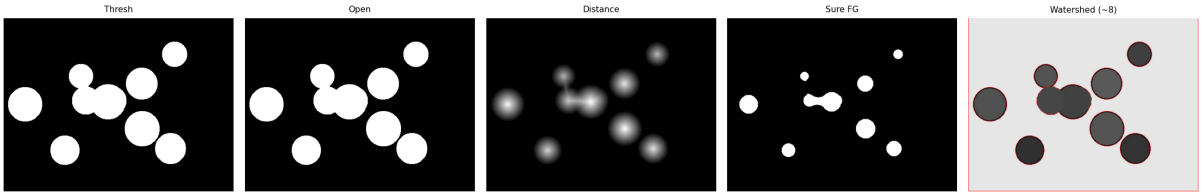
- t1_threshold.png, t1_handmade_mask.png, t1_iou.csv.
- t2_watershed.png — distance transform story.
- t3_grabcut.png — iteration comparison.
- t4_deeplab.png, t4_maskrcnn.png, t4_semantic_vs_instance.png.



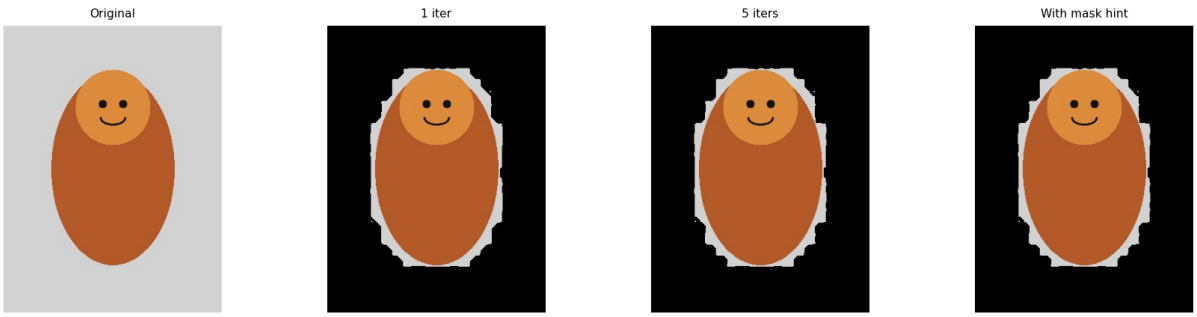
t1_handmade_mask.png



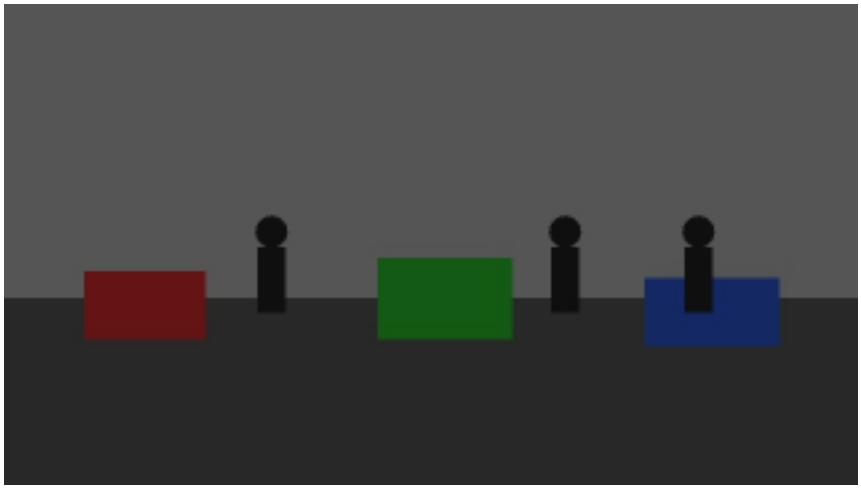
t1_threshold.png



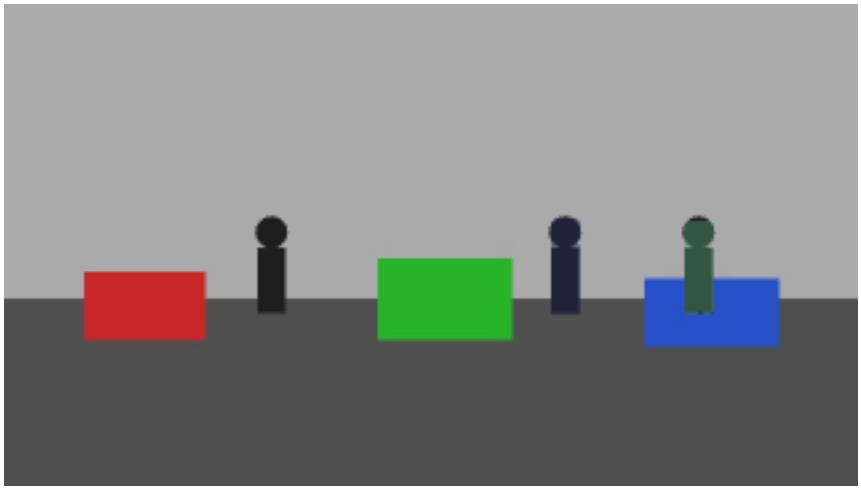
t2_watershed.png



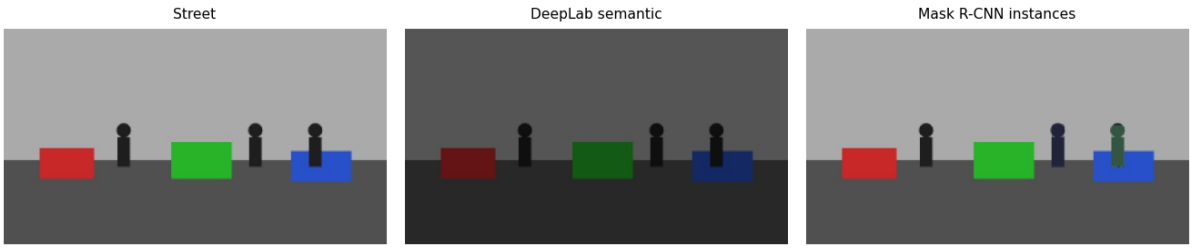
t3_grabcut.png



t4_deeplab.png



t4_maskrcnn.png



t4_semantic_vs_instance.png

Practical 6 — Motion, Tracking, Geometry

Why this practical exists

Video is a stack of images plus time. Optical flow estimates motion; Kalman filters smooth noisy measurements; calibration undistorts lenses; AR overlays 3D points; stereo estimates depth. This is where CV meets physics and geometry.

Task-by-task (baby steps)

Task 1 — Optical flow

Lucas-Kanade tracks sparse corner points frame-to-frame. Farneback gives dense flow coloured by direction/magnitude. Brightness constancy breaks when lighting jumps — we show that failure.

Task 2 — Kalman filter tracking

Detect red blob centroid (measurement), predict with constant-velocity model, update when seen, coast when occluded. Compare to CSRT tracker if available.

Task 3 — Camera calibration

Multiple chessboard views → intrinsic matrix K and distortion coeffs → undistort. Report RMS reprojection error in pixels.

Task 4 — AR overlay

solvePnP gives pose; projectPoints draws a 3D cube on the board plane.

Task 5 — Stereo depth

Semi-global block matching on a rectified pair; disparity map brightness ≈ nearer objects.

Core formula / snippet to know

```
# Predict (constant-velocity model):
x = F @ x
P = F @ P @ F.T + Q
# Update when measurement z arrives:
K = P @ H.T @ inv(H @ P @ H.T + R)
x = x + K @ (z - H @ x)
P = (I - K @ H) @ P
# If z is None (occlusion): skip update, coast on prediction
```

How marks usually work

- LK trail figure + Farneback colour flow.
- Kalman video/plot showing coast through occlusion.
- Calibration RMS + undistort before/after.
- AR cube on chessboard.
- Stereo disparity with parameter note.

How to run

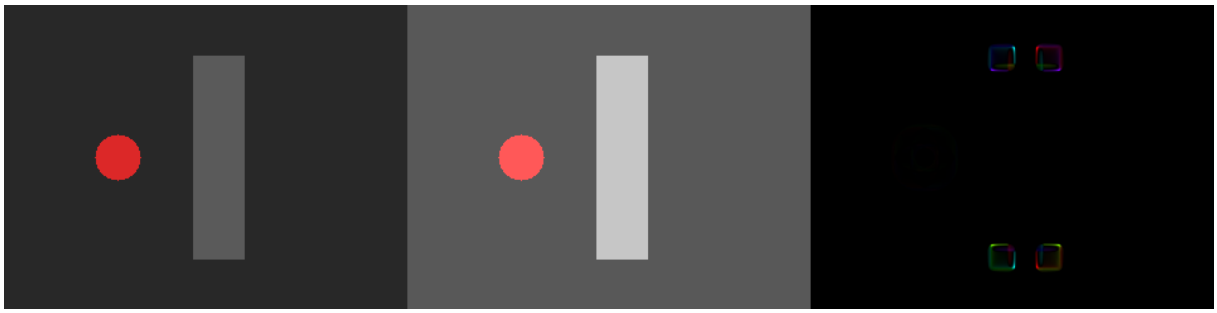
```
python prac6/run_prac6.py
```

Figures to inspect

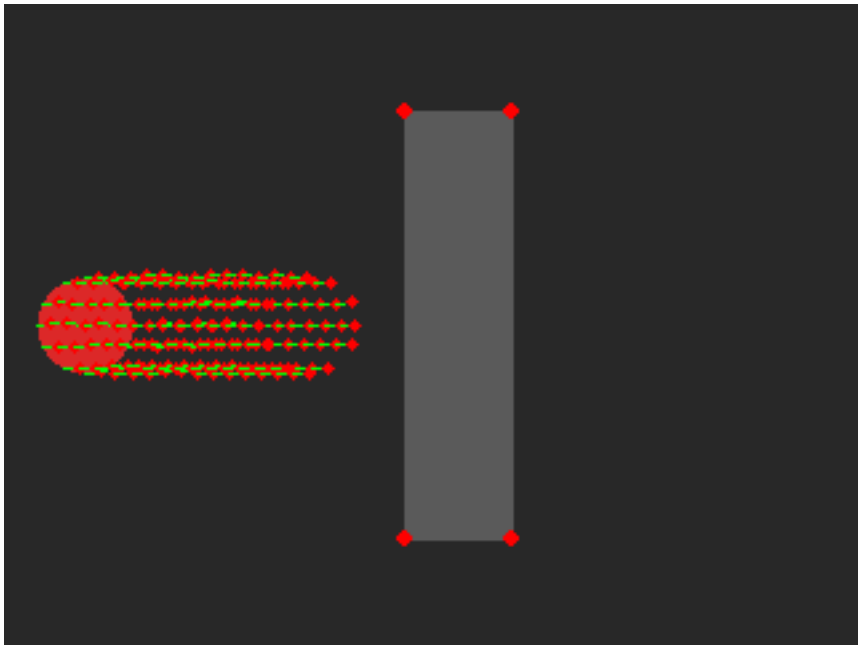
- t1_lk_trails.png, t1_farneback.png, t1_flow_failure.png.
- t2_kalman.avi, t2_kf_plot.png — tracking story.
- t3_undistort.png, t3_calib.txt.
- t4_ar_cube.png.
- t5_stereo.png, t5_disp_nd*.png.



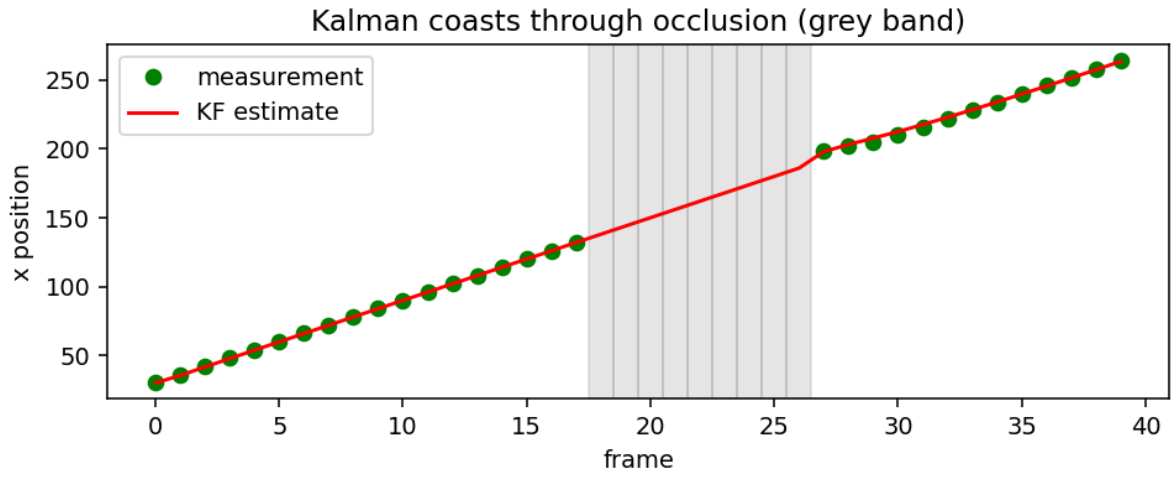
t1_farneback.png



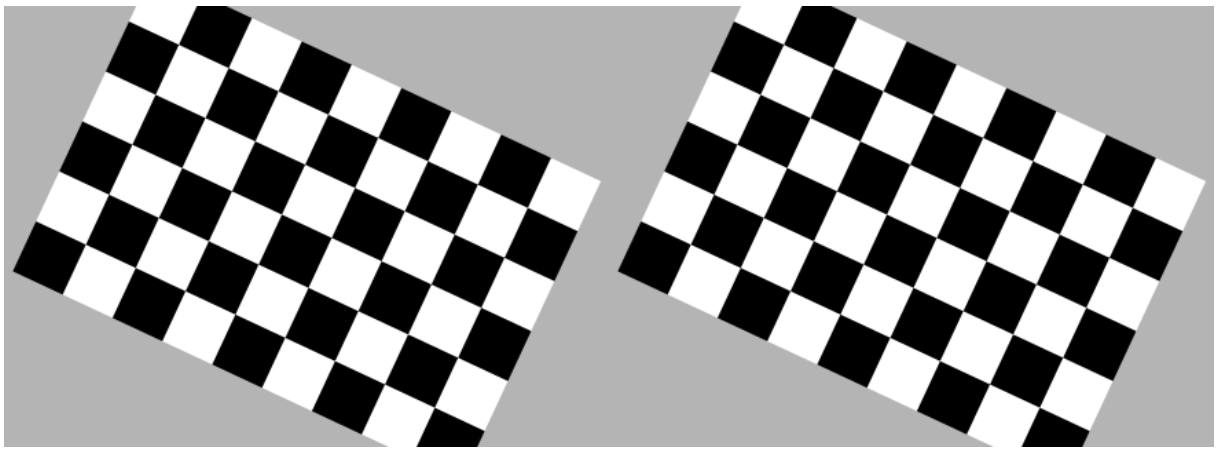
t1_flow_failure.png



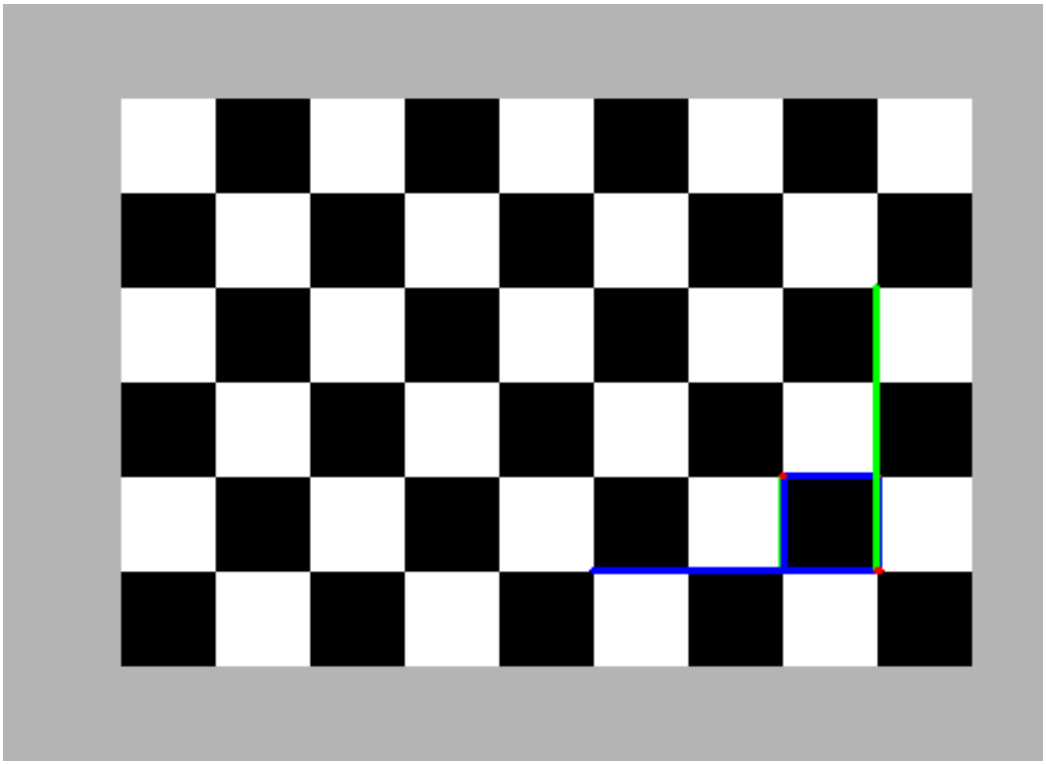
t1_lk_trails.png



t2_kf_plot.png



t3_undistort.png



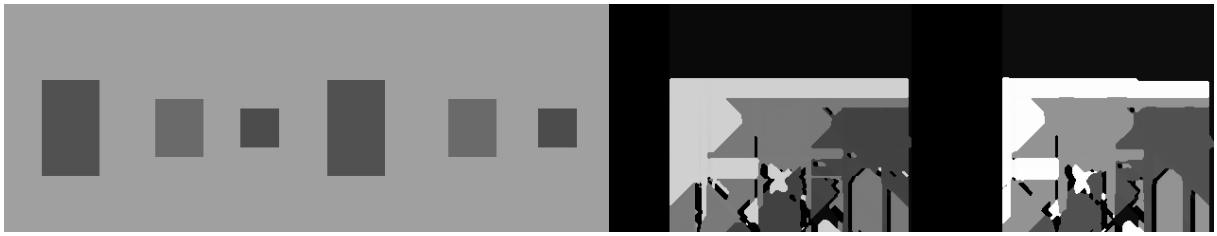
t4_ar_cube.png



t5_disp_nd64_bs5.png



t5_disp_nd96_bs9.png



t5_stereo.png

Practical 7 — Capstone — Deployed Application

Why this practical exists

Everything merges: classification from Prac 4, Gradio UI from Prac 3/4/5, tests + CI from software engineering expectations. My track is agriculture — maize leaf triage for smallholders who need a screening hint, not a fake doctor.

Task-by-task (baby steps)

Problem + ethics

Wrong fungicide advice costs money; missed rust spreads. UI must scream disclaimer and prefer 'unsure' when confidence is low.

Model + fallback

Load `prac4/model.pt` if present; else colour heuristic so the app still runs in marking.

Quality gates

Blur check rejects shaky photos. Confidence slider lets user tune how cautious the system is.

Engineering

`pytest` on preprocess shape; GitHub Actions workflow stub; HF Space deploy with your token.

Core formula / snippet to know

```
# Capstone predict flow:
# probs = model(image) or heuristic(image)
# if max(probs) < conf_gate: return 'Unsure'
# else: return class + guidance text
```

How marks usually work

- Working Gradio app with disclaimer and confidence gate.
- Tests pass in CI.
- Proposal aligns with implemented scope (or document gap).
- HF Space public URL + GitHub repo with sensible commit history.
- Team split documented if pair project.

How to run

```
python prac4/train_prac4.py   # optional weights
cd prac7 && pytest -q && cd ..
python prac7/app.py
```

Figures to inspect

- No standard figure script — screenshot your Gradio UI and ethics page for the report.
- Reuse Prac 4 Grad-CAM figures as evidence the model looks at leaves.

No figures folder yet — run the practical script first: C:\Users\msiny\Documents\school stuff\csc6302-practicals\prac7\report\figures

Capstone proposal summary — Agriculture (maize leaf triage)

Problem statement

Smallholder farmers and students need a phone-first screening aid: photograph a maize leaf, get a suggestion of healthy vs rust vs blight, with confidence and a hard disclaimer that this is not a diagnosis. Zambia's agriculture extension services are thin; a bad automated guess is worse than saying 'I am not sure'.

Dataset plan

- Preferred: real field photos around Lusaka / UNZA farms — consent, no identifiable people.
- Scaffold fallback: tiny synthetic set under `shared/images/leaf_dataset` (70/15/15 split).
- Evaluation: ≥30 held-out test images; report accuracy and per-class F1.

Model plan

- Fine-tune MobileNetV3-Small (Practical 4 pipeline): freeze → train head → light fine-tune.
- Optional HSV pre-check: is this even green / leaf-ish?
- Optional VLM explanation via OpenRouter free tier with template fallback on rate limits.

Team split (template)

Member A	Data collection, training notebook, metrics
Member B	Gradio app, tests, CI/CD, ethics page

Risks

False blight alarms waste spray money; missed rust can spread. We tune the confidence gate toward 'unsure' rather than a confident wrong class. Partner still TBA — register with lecturer by the gate date.

What I still must do myself (honest checklist)

This repo is a scaffold, not a submitted soul. Before I claim full marks, I need to tick these myself — no AI shortcut replaces them.

- Replace every synthetic demo image with my own phone photos (colour scene, coins, panorama pair, group shot, document, portrait, motion clip, stereo pair, leaf photos).
- Re-run all figure scripts on my images; rewrite report paragraphs so they describe what I actually see.
- Colab GPU run for CIFAR-10 (or full leaf dataset) with enough epochs — CPU demo numbers alone are weak evidence.
- Create Hugging Face Spaces for Prac 3/4/7 (and any other required N); set HF_TOKEN in GitHub Actions secrets for deploy.
- Register capstone partner with lecturer; create shared repo csc6302-capstone-SINYANI-PARTNER with real commit history from both members.
- Collect ≥30 real maize leaf test photos with labels I trust (or extension officer verification notes).
- Write AI declaration in each practical README and report — what I used, what I verified line-by-line.
- Practice viva explanations for PSNR, Sobel loops, Harris R, IoU, Kalman update without reading slides.
- Submit PDF reports via LMS with correct SINYANI naming; double-check deadline timezone.

If a figure folder is empty when you read this PDF, that only means I have not run the generator yet — not that the practical is optional. Run the script, then rebuild this PDF.